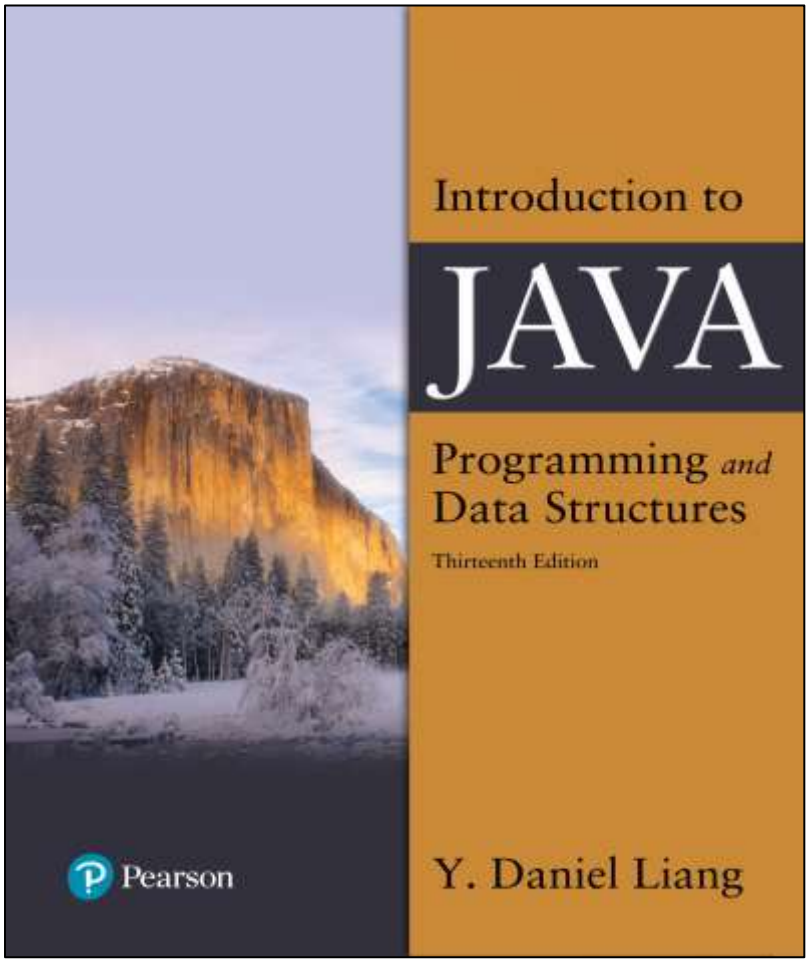
Introduction to Java Programming and Data Structures

Thirteenth Edition



Chapter 18

Recursion

Pearson

Copyright © 2024 Pearson Education, Inc. All Rights Reserved

Motivations (1 of 2)

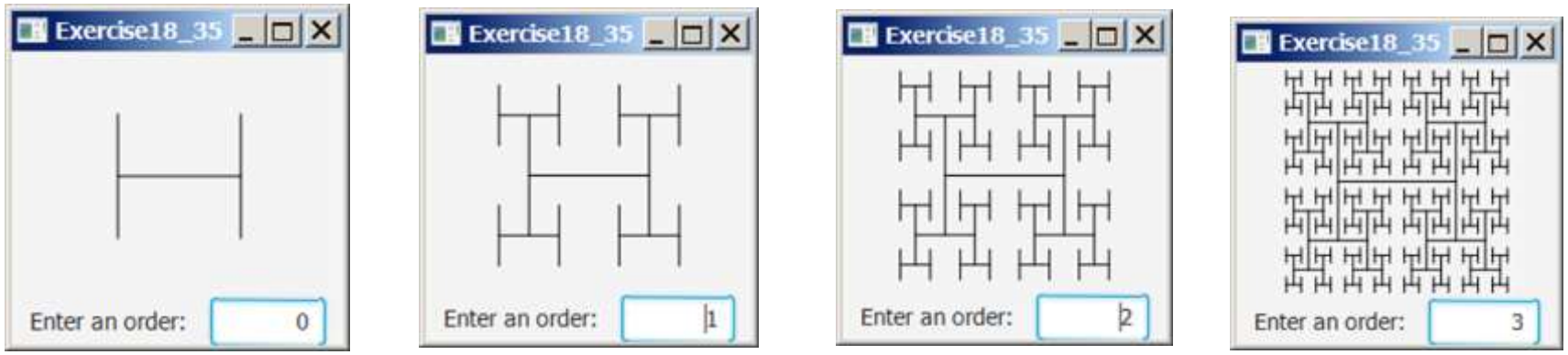
Suppose you want to find all the files under a directory that contains a particular word. How do you solve this problem? There are several ways to solve this problem. An intuitive solution is to use recursion by searching the files in the subdirectories recursively.

Pearson

Copyright © 2024 Pearson Education, Inc. All Rights Reserved

Motivations (2 of 2)

H-trees, depicted in Figure 18.1, are used in a very large-scale integration (VLSI) design as a clock distribution network for routing timing signals to all parts of a chip with equal propagation delays. How do you write a program to display H-trees? A good approach is to use recursion.



Pearson

Copyright © 2024 Pearson Education, Inc. All Rights Reserved

Objectives (1 of 2)

18.1 To describe what a recursive method is and the benefits of using recursion (§18.1).

18.2 To develop recursive methods for recursive mathematical functions (§§18.2–18.3).

18.3 To explain how recursive method calls are handled in a call stack (§§18.2–18.3).

18.4 To solve problems using recursion (§18.4).

18.5 To use an overloaded helper method to derive a recursive method (§18.5).

18.6 To implement a selection sort using recursion (§18.5.1).

Pearson

Copyright © 2024 Pearson Education, Inc. All Rights Reserved

Objectives (2 of 2)

18.7 To implement a binary search using recursion (§18.5.2).

18.8 To get the directory size using recursion (§18.6).

18.9 To solve the Tower of Hanoi problem using recursion (§18.7).

18.10 To draw fractals using recursion (§18.8).

18.11 To discover the relationship and difference between recursion and iteration (§18.9).

18.12 To know tail-recursive methods and why they are desirable (§18.10).

Pearson

Copyright © 2024 Pearson Education, Inc. All Rights Reserved

Computing Factorial (1 of 11)

Mathematic notation:

$$n! = n * (n-1)!, \ n > 0$$

$$0! = 1$$

Function:

$$\text{factorial}(0) = 1;$$

$$\text{factorial}(n) = n * \text{factorial}(n-1); \ n > 0$$

ComputeFactorial

Pearson

Copyright © 2024 Pearson Education, Inc. All Rights Reserved

Computing Factorial (2 of 11)

$$\text{factorial}(0) = 1;$$
$$\text{factorial}(n) = n*\text{factorial}(n-1);$$

factorial(4)

Pearson

Copyright © 2024 Pearson Education, Inc. All Rights Reserved

Computing Factorial (3 of 11)

$$\text{factorial}(0) = 1;$$
$$\text{factorial}(n) = n*\text{factorial}(n-1);$$

$$\text{factorial}(4) = 4 * \text{factorial}(3)$$

Pearson

Copyright © 2024 Pearson Education, Inc. All Rights Reserved

Computing Factorial (4 of 11)

$$\text{factorial}(0) = 1;$$
$$\text{factorial}(n) = n*\text{factorial}(n-1);$$

$$\text{factorial}(4) = 4 * \text{factorial}(3)$$
$$= 4 * 3 * \text{factorial}(2))$$

Pearson

Copyright © 2024 Pearson Education, Inc. All Rights Reserved

Computing Factorial (5 of 11)

$$\text{factorial}(0) = 1;$$
$$\text{factorial}(n) = n*\text{factorial}(n-1);$$

$$\begin{aligned}\text{factorial}(4) &= 4 * \text{factorial}(3)\\ &= 4 * 3 * \text{factorial}(2)\\ &= 4 * 3 * (2 * \text{factorial}(1))\end{aligned}$$

Pearson

Copyright © 2024 Pearson Education, Inc. All Rights Reserved

Computing Factorial (6 of 11)

$$\text{factorial}(0) = 1;$$
$$\text{factorial}(n) = n*\text{factorial}(n-1);$$

$$\begin{aligned}\text{factorial}(4) &= 4 * \text{factorial}(3)\\ &= 4 * 3 * \text{factorial}(2)\\ &= 4 * 3 * (2 * \text{factorial}(1))\\ &= 4 * 3 * (2 * (1 * \text{factorial}(0)))\end{aligned}$$

Pearson

Copyright © 2024 Pearson Education, Inc. All Rights Reserved

Computing Factorial (7 of 11)

$$\text{factorial}(0) = 1;$$
$$\text{factorial}(n) = n*\text{factorial}(n-1);$$

$$\begin{aligned}
\text{factorial}(4) &= 4 * \text{factorial}(3) \\
&= 4 * 3 * \text{factorial}(2) \\
&= 4 * 3 * (2 * \text{factorial}(1)) \\
&= 4 * 3 * (2 * (1 * \text{factorial}(0))) \\
&= 4 * 3 * (2 * (1 * 1)))
\end{aligned}$$

Pearson

Copyright © 2024 Pearson Education, Inc. All Rights Reserved

Computing Factorial (8 of 11)

$$\text{factorial}(0) = 1;$$
$$\text{factorial}(n) = n*\text{factorial}(n-1);$$

$$\begin{aligned}
\text{factorial}(4) &= 4 * \text{factorial}(3) \\
&= 4 * 3 * \text{factorial}(2) \\
&= 4 * 3 * (2 * \text{factorial}(1)) \\
&= 4 * 3 * (2 * (1 * \text{factorial}(0))) \\
&= 4 * 3 * (2 * (1 * 1))) \\
&= 4 * 3 * (2 * 1)
\end{aligned}$$

Pearson

Copyright © 2024 Pearson Education, Inc. All Rights Reserved

Computing Factorial (9 of 11)

$$\text{factorial}(0) = 1;$$
$$\text{factorial}(n) = n*\text{factorial}(n-1);$$

$$\begin{aligned}
\text{factorial}(4) &= 4 * \text{factorial}(3) \\
&= 4 * 3 * \text{factorial}(2) \\
&= 4 * 3 * (2 * \text{factorial}(1)) \\
&= 4 * 3 * (2 * (1 * \text{factorial}(0))) \\
&= 4 * 3 * (2 * (1 * 1))) \\
&= 4 * 3 * (2 * 1) \\
&= 4 * 3 * 2
\end{aligned}$$

Pearson

Copyright © 2024 Pearson Education, Inc. All Rights Reserved

Computing Factorial (10 of 11)

$$\text{factorial}(0) = 1;$$
$$\text{factorial}(n) = n*\text{factorial}(n-1);$$

$$\begin{aligned}
\text{factorial}(4) &= 4 * \text{factorial}(3) \\
&= 4 * (3 * \text{factorial}(2)) \\
&= 4 * (3 * (2 * \text{factorial}(1))) \\
&= 4 * (3 * (2 * (1 * \text{factorial}(0)))) \\
&= 4 * (3 * (2 * (1 * 1))) \\
&= 4 * (3 * (2 * 1)) \\
&= 4 * (3 * 2) \\
&= 4 * (6)
\end{aligned}$$

Pearson

Copyright © 2024 Pearson Education, Inc. All Rights Reserved

Computing Factorial (11 of 11)

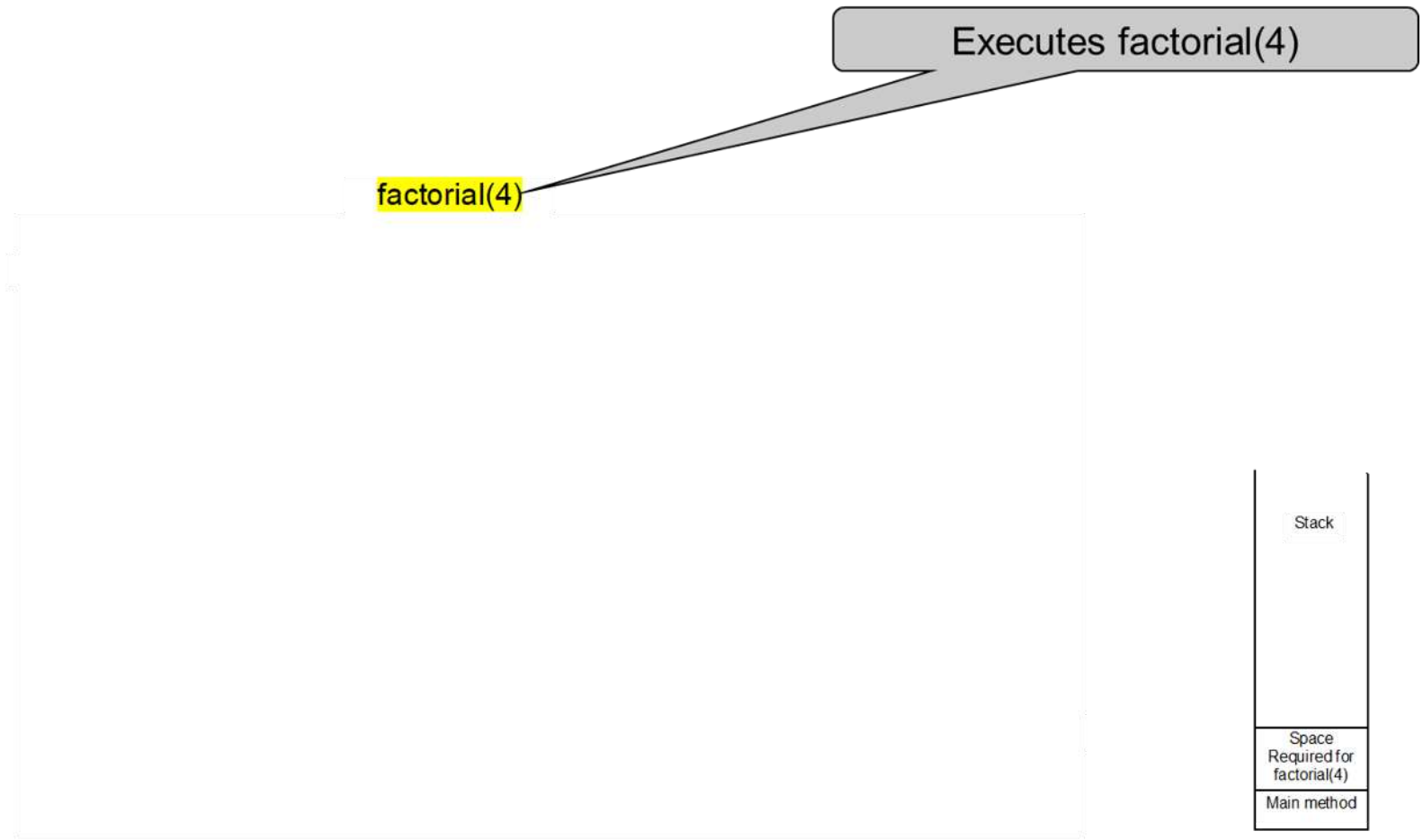
$$\text{factorial}(0) = 1;$$
$$\text{factorial}(n) = n*\text{factorial}(n-1);$$

$$\begin{aligned}
\text{factorial}(4) &= 4 * \text{factorial}(3) \\
&= 4 * (3 * \text{factorial}(2)) \\
&= 4 * (3 * (2 * \text{factorial}(1))) \\
&= 4 * (3 * (2 * (1 * \text{factorial}(0)))) \\
&= 4 * (3 * (2 * (1 * 1))) \\
&= 4 * (3 * (2 * 1)) \\
&= 4 * (3 * 2) \\
&= 4 * (6) \\
&= 24
\end{aligned}$$

Pearson

Copyright © 2024 Pearson Education, Inc. All Rights Reserved

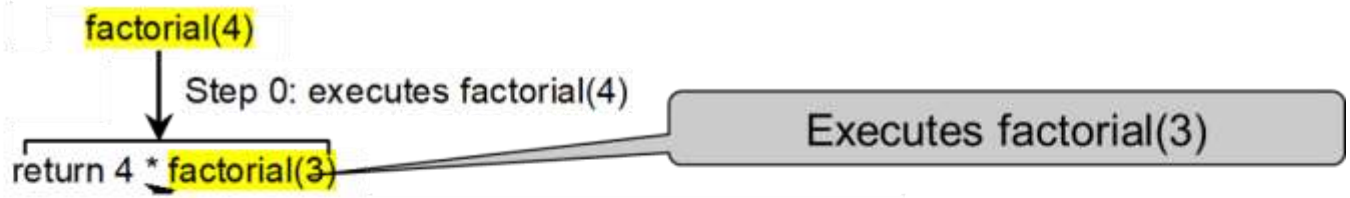
Trace Recursive Factorial (1 of 11)

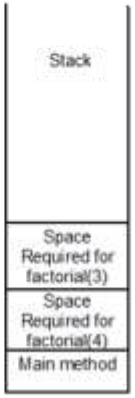


Pearson

Copyright © 2024 Pearson Education, Inc. All Rights Reserved

Trace Recursive Factorial (2 of 11)

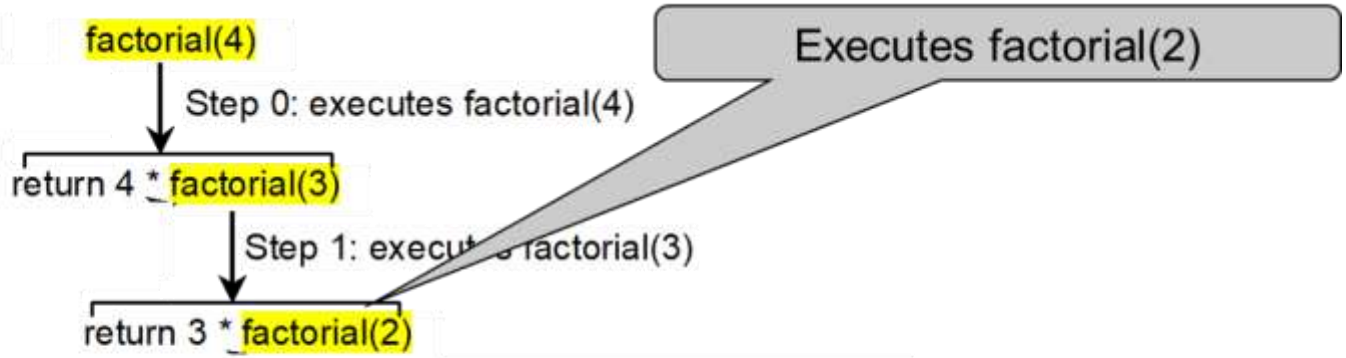


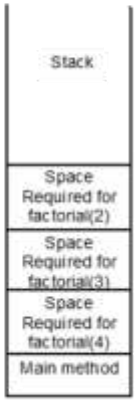


Pearson

Copyright © 2024 Pearson Education, Inc. All Rights Reserved

Trace Recursive Factorial (3 of 11)

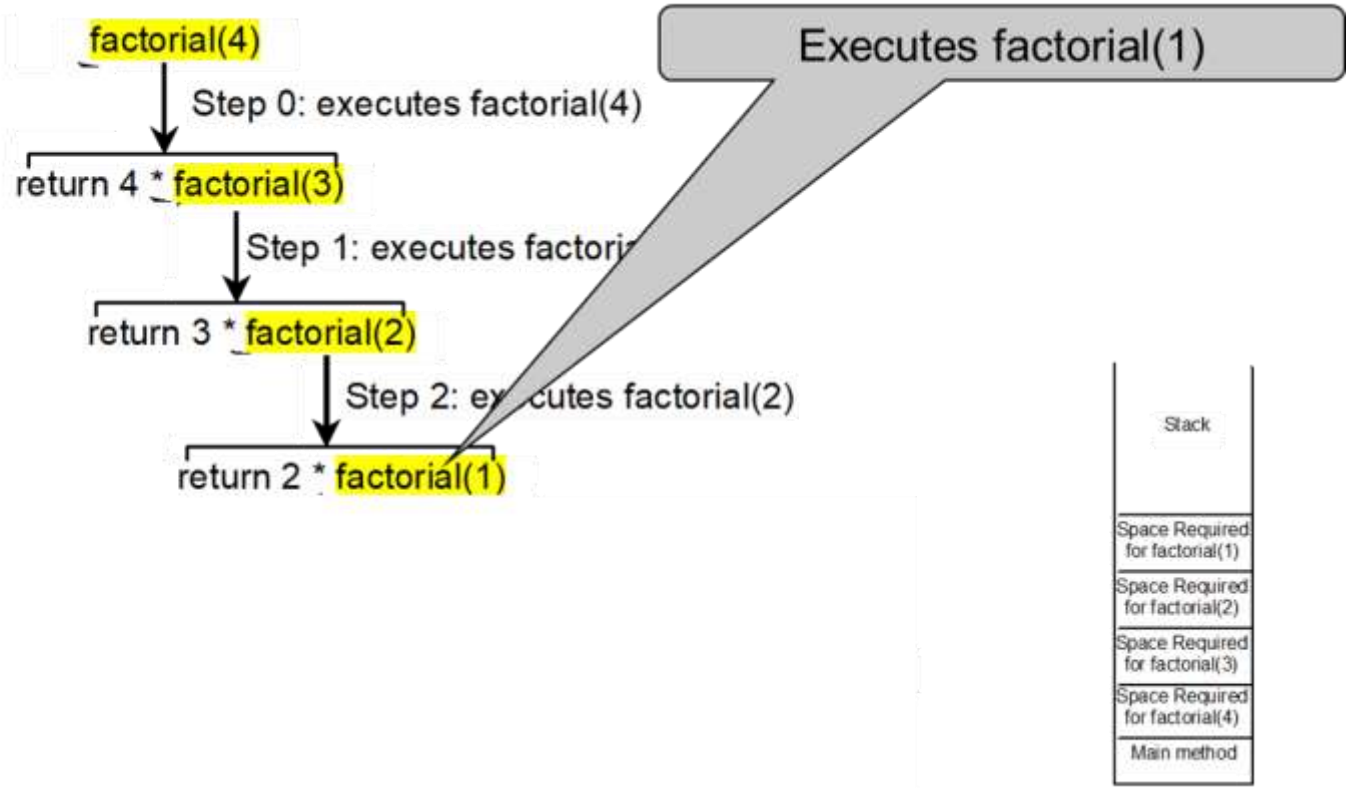




Pearson

Copyright © 2024 Pearson Education, Inc. All Rights Reserved

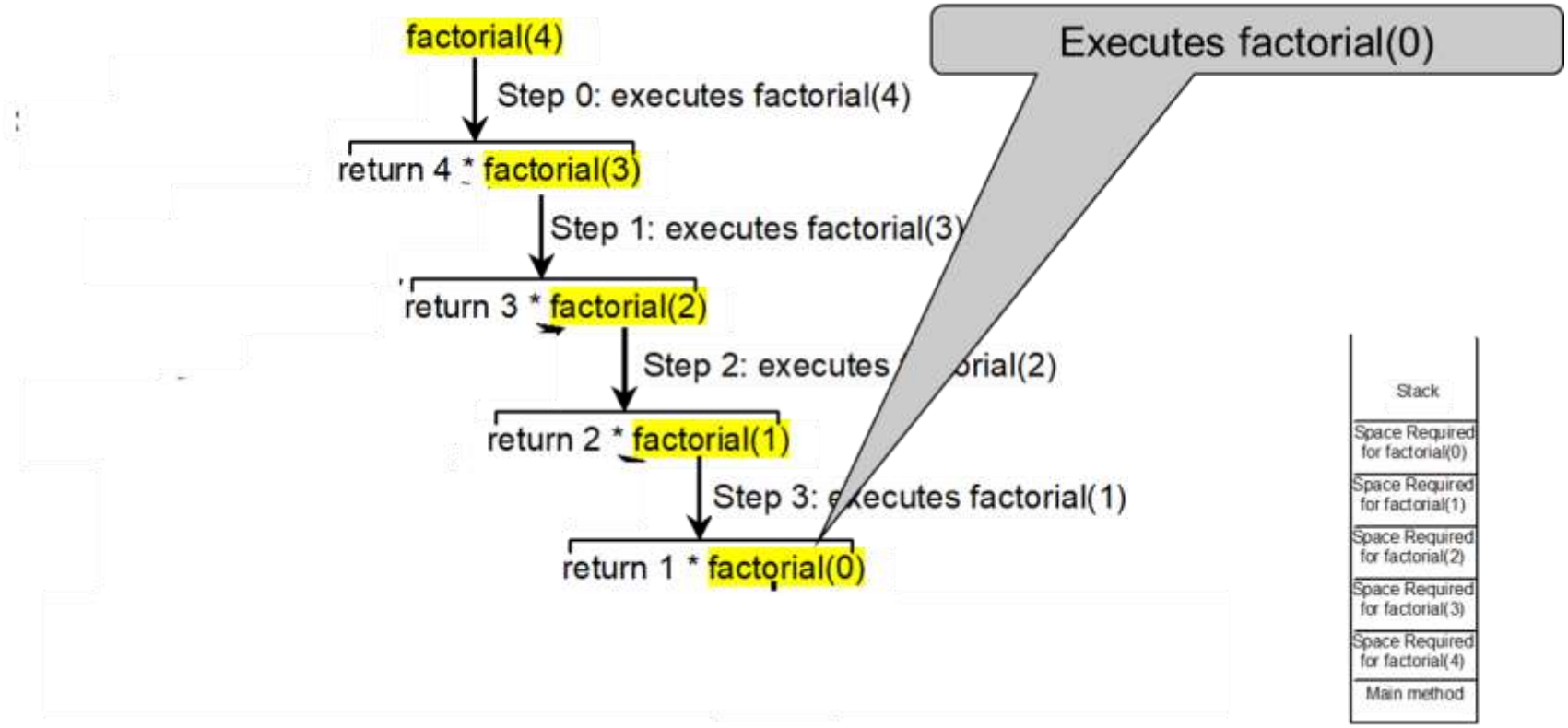
Trace Recursive Factorial (4 of 11)



Pearson

Copyright © 2024 Pearson Education, Inc. All Rights Reserved

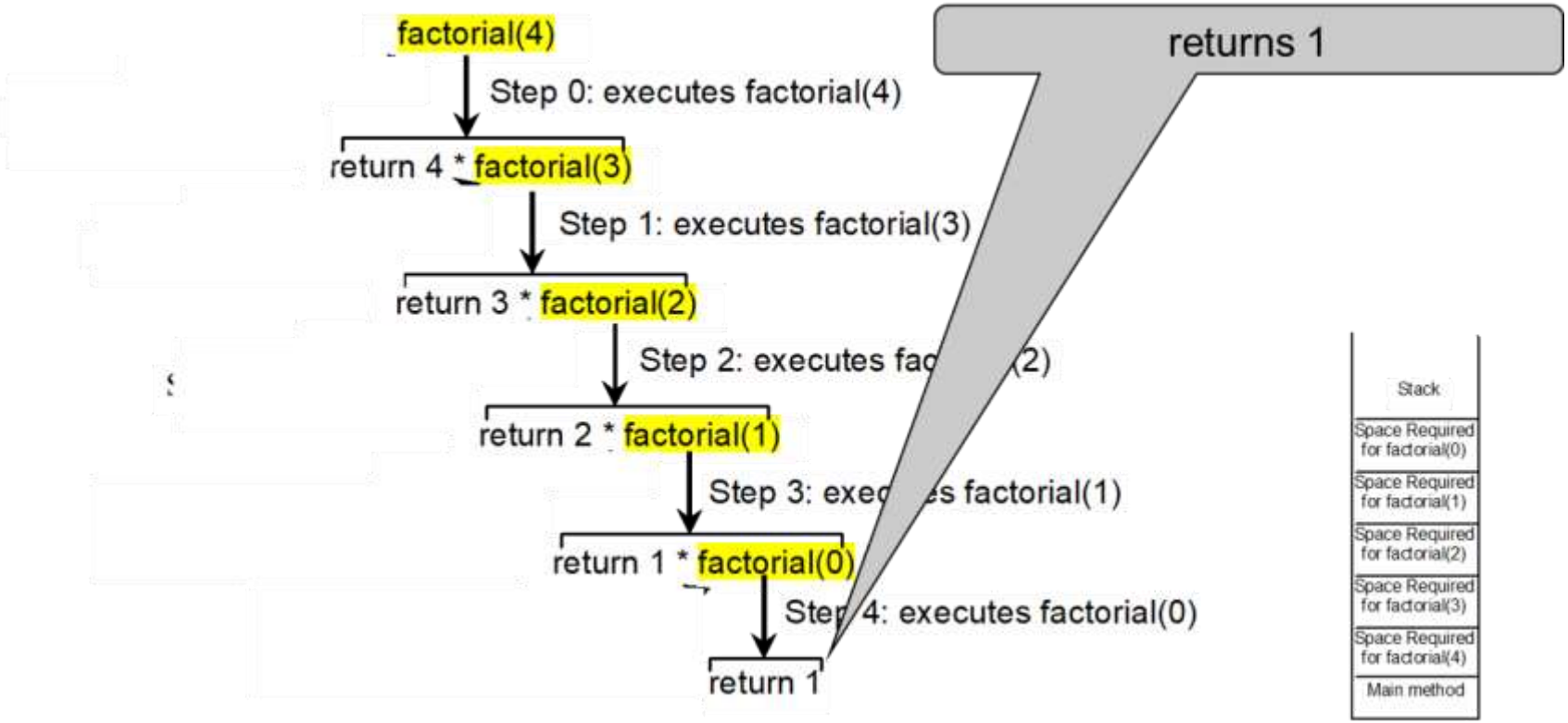
Trace Recursive Factorial (5 of 11)



Pearson

Copyright © 2024 Pearson Education, Inc. All Rights Reserved

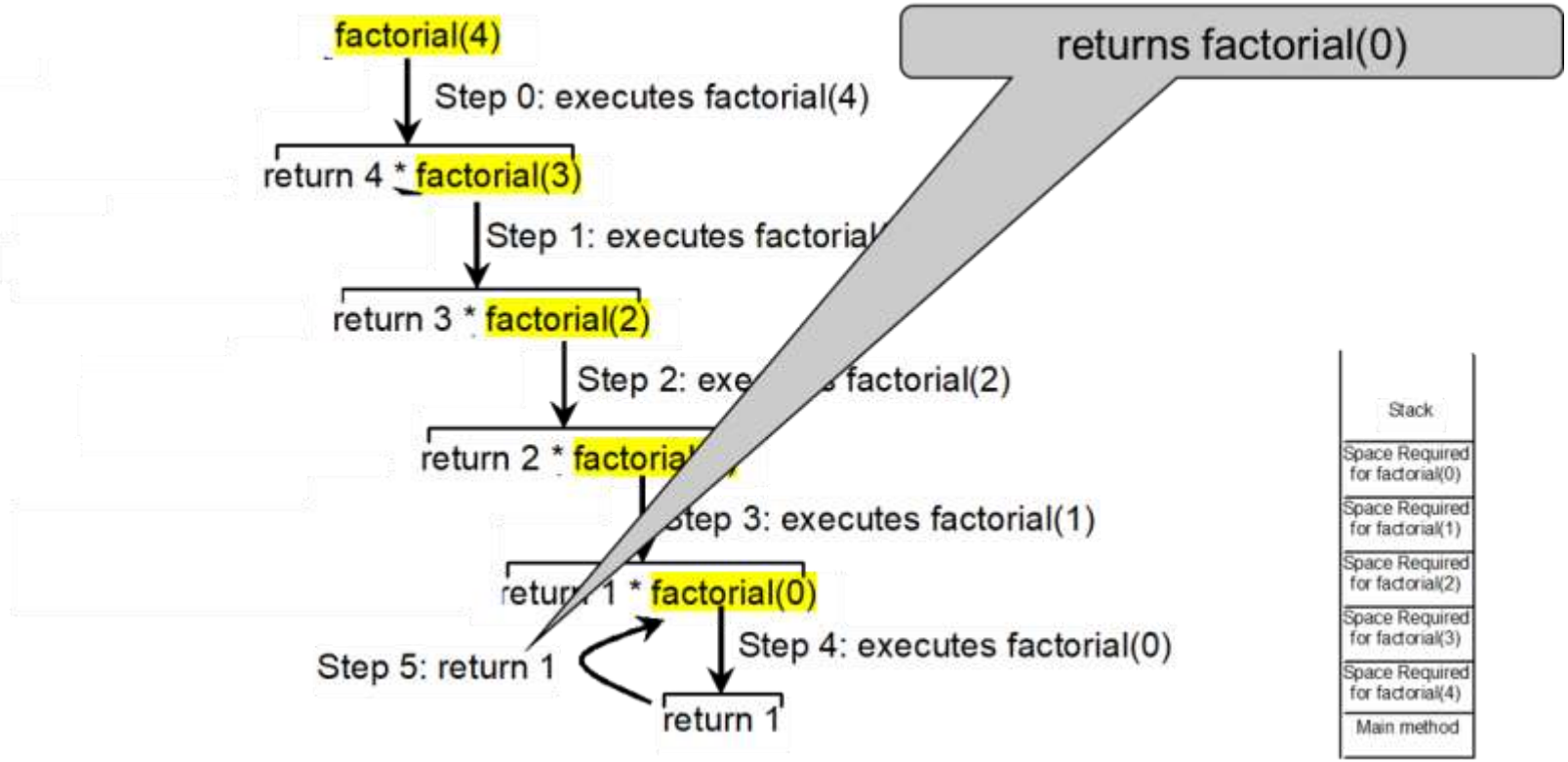
Trace Recursive Factorial (6 of 11)



Pearson

Copyright © 2024 Pearson Education, Inc. All Rights Reserved

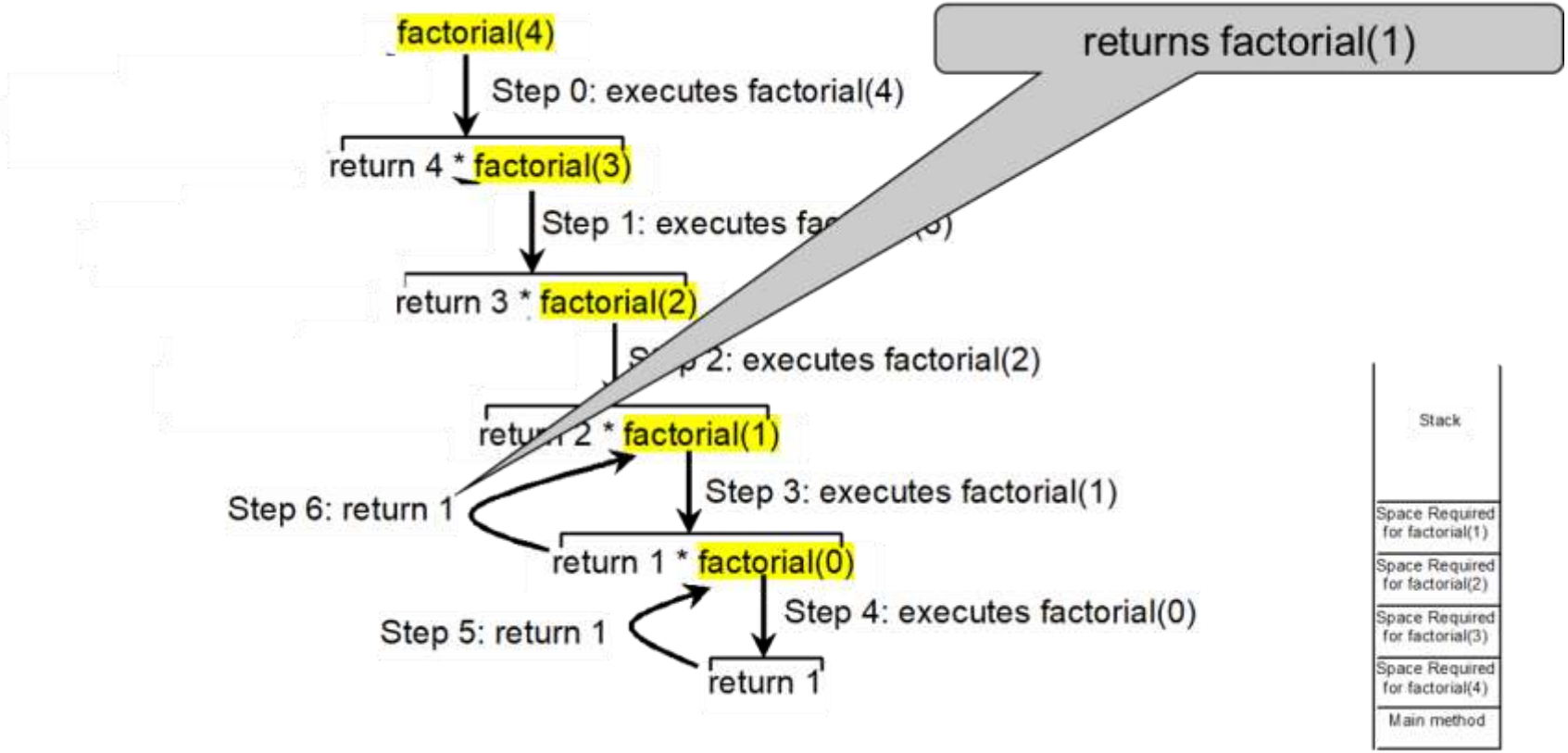
Trace Recursive Factorial (7 of 11)



Pearson

Copyright © 2024 Pearson Education, Inc. All Rights Reserved

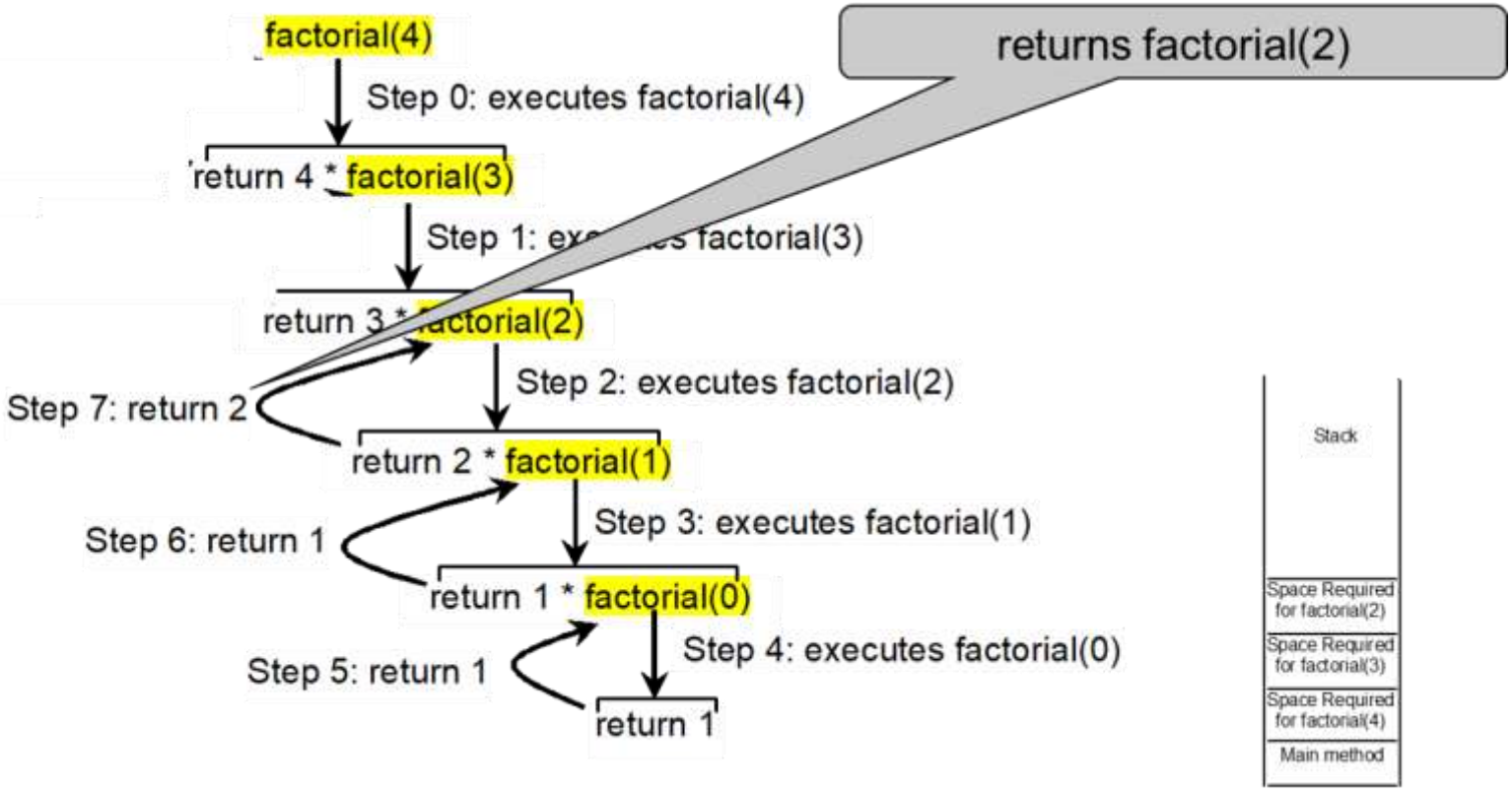
Trace Recursive Factorial (8 of 11)



Pearson

Copyright © 2024 Pearson Education, Inc. All Rights Reserved

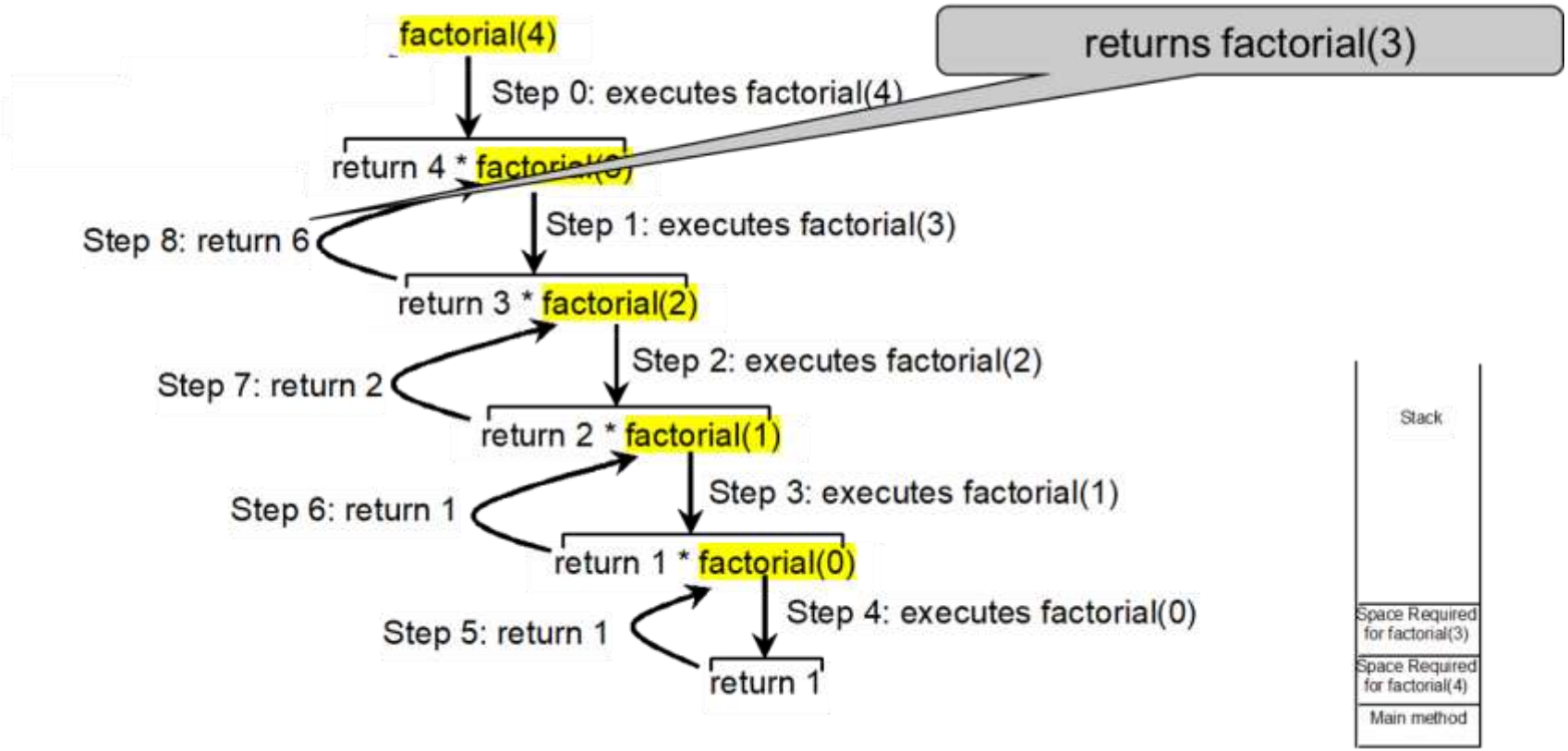
Trace Recursive Factorial (9 of 11)



Pearson

Copyright © 2024 Pearson Education, Inc. All Rights Reserved

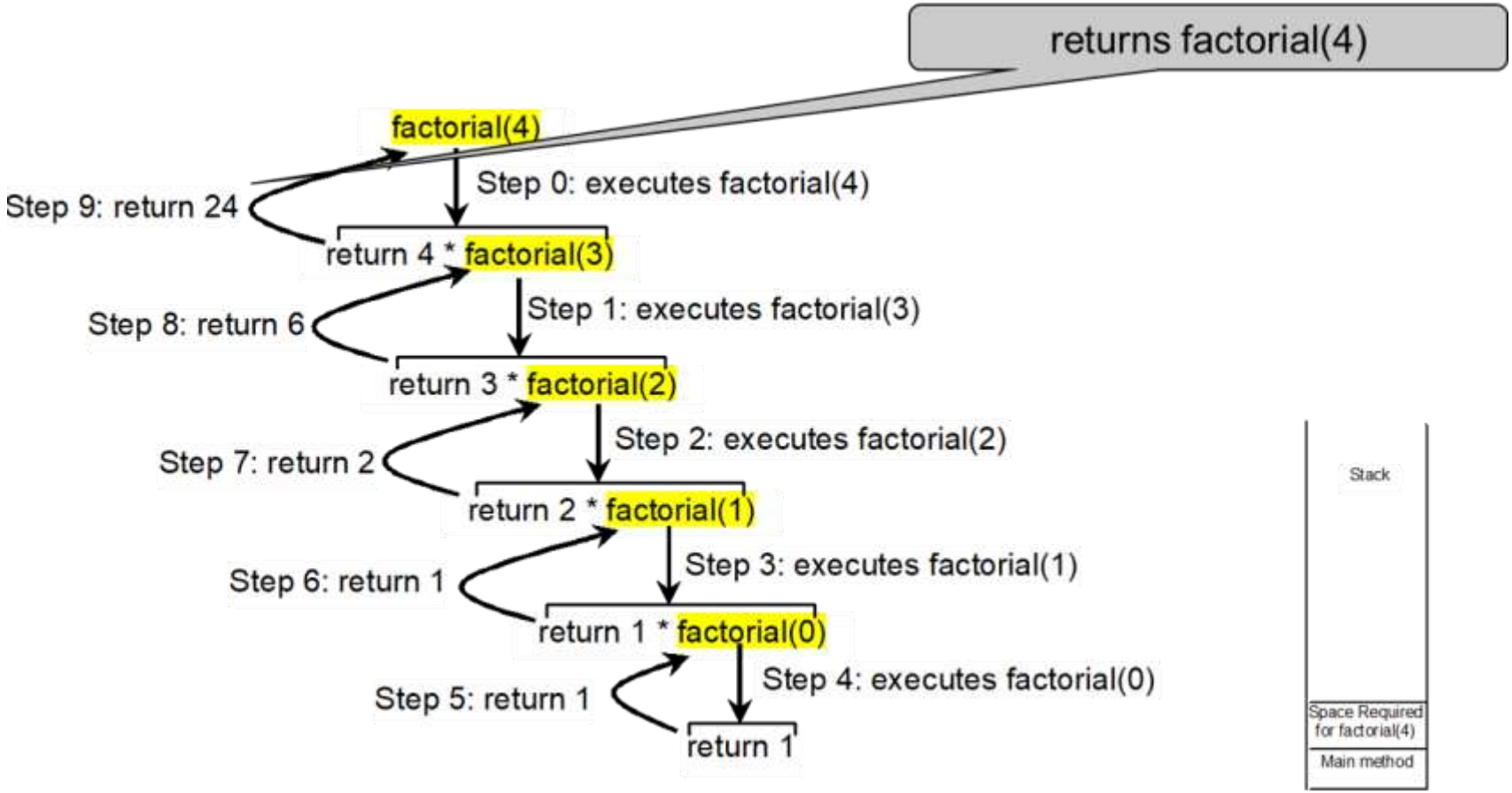
Trace Recursive Factorial (10 of 11)



Pearson

Copyright © 2024 Pearson Education, Inc. All Rights Reserved

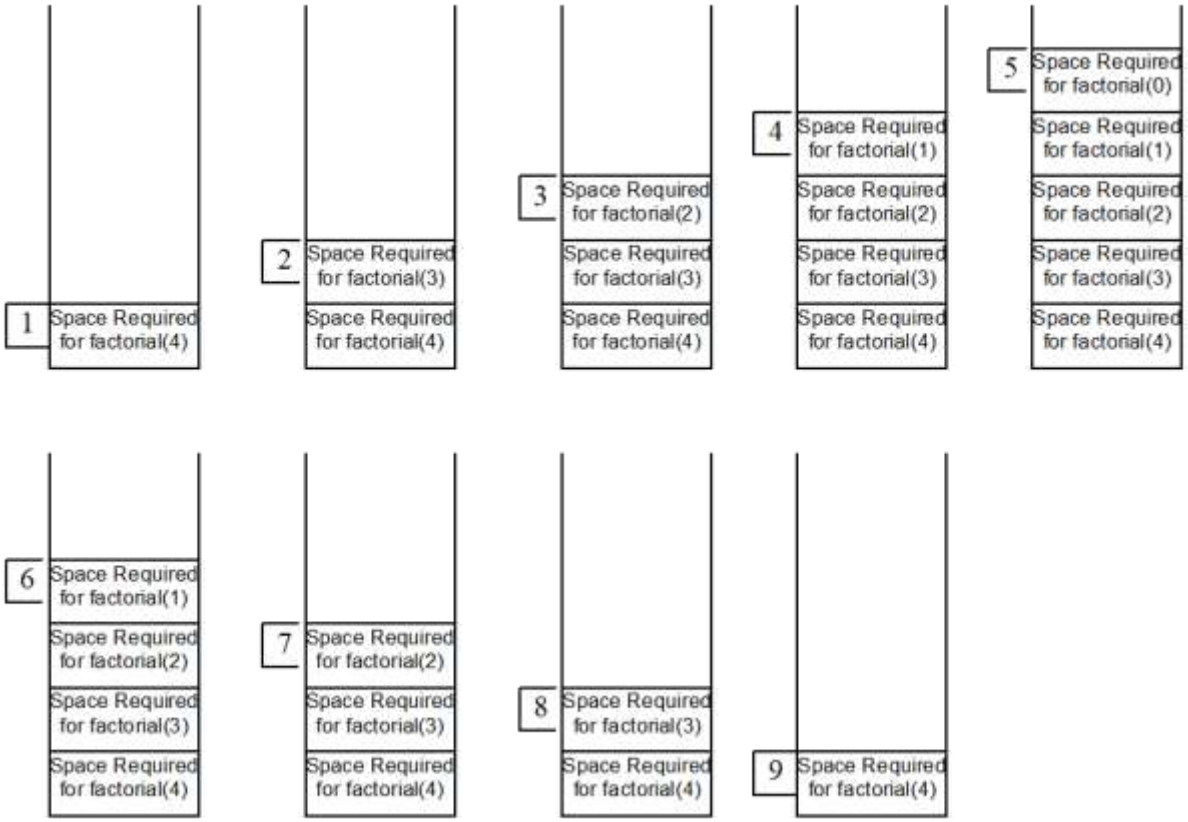
Trace Recursive Factorial (11 of 11)



Pearson

Copyright © 2024 Pearson Education, Inc. All Rights Reserved

factorial(4) Stack Trace



Pearson

Copyright © 2024 Pearson Education, Inc. All Rights Reserved

Other Examples

$$f(0) = 0;$$

$$f(n) = n + f(n-1);$$

Pearson

Copyright © 2024 Pearson Education, Inc. All Rights Reserved

Fibonacci Numbers (1 of 2)

```
Fibonacci series: 0 1 1 2 3 5 8 13 21 34 55
89…

indices: 0 1 2 3 4 5 6 7 8 9 10 11
```

$$\text{fib}(0) = 0;$$

$$\text{fib}(1) = 1;$$

$$\text{fib}(\text{index}) = \text{fib}(\text{index} - 1) + \text{fib}(\text{index} - 2); \text{index} >= 2$$

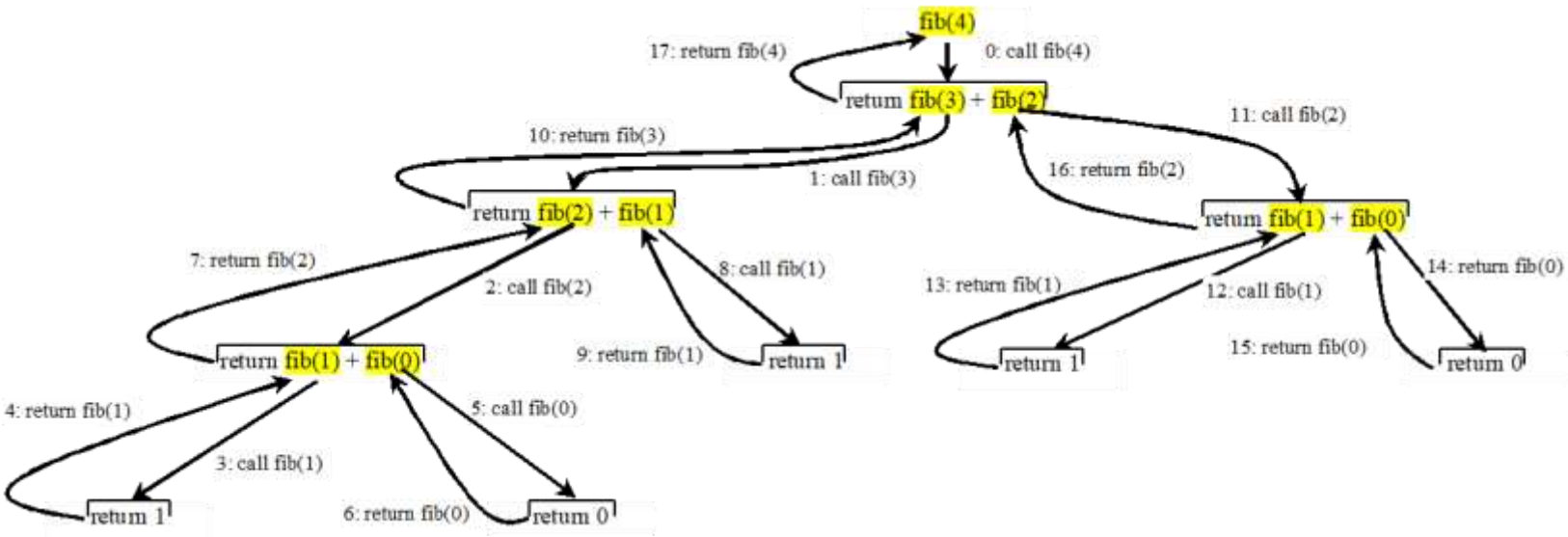
$$\text{fib}(3) = \text{fib}(2) + \text{fib}(1) = (\text{fib}(1) + \text{fib}(0)) + \text{fib}(1) = (1 + 0)$$
$$+\text{fib}(1) = 1 + \text{fib}(1) = 1 + 1 = 2$$

ComputeFibonacci

Pearson

Copyright © 2024 Pearson Education, Inc. All Rights Reserved

Fibonacci Numbers (2 of 2)



Pearson

Copyright © 2024 Pearson Education, Inc. All Rights Reserved

Problem Solving Using Recursion (1 of 2)

In general, to solve a problem using recursion, you break it into subproblems. If a subproblem resembles the original problem, you can apply the same approach to solve the subproblems recursively. A subproblem is almost the same as the original problem in nature with a smaller size.

Pearson

Copyright © 2024 Pearson Education, Inc. All Rights Reserved

Characteristics of Recursion

- All recursive methods have the following characteristics:
  - The method is implemented using a conditional statement that leads to different cases.
  - One or more base cases (the simplest case) are used to stop recursion.
  - Every recursive call reduces the original problem, bringing it increasingly closer to a base case until it becomes that case.

Pearson

Copyright © 2024 Pearson Education, Inc. All Rights Reserved

Problem Solving Using Recursion (2 of 2)

`nPrintln("Welcome", n);`

1. one is to print the message one time and the other is to print the message for `n-1` times.
2. The second problem is the same as the original problem with a smaller size.
3. The base case for the problem is `n==0`. You can solve this problem using recursion as follows:

```
public static void nPrintln(String message, int n) {
  if (n >= 1) {
    System.out.println(message);
    nPrintln(message, n - 1);
  } // The base case is n < 1
}
```

Pearson

Copyright © 2024 Pearson Education, Inc. All Rights Reserved

Think Recursively

Many of the problems presented in the early chapters can be solved using recursion if you **think recursively**. For example, the palindrome problem can be solved recursively as follows:

```
public static boolean isPalindrome(String s) {
  if (s.length() <= 1) // Base case
    return true;
  else if (s.charAt(0) != s.charAt(s.length() - 1)) // Base case
    return false;
  else
    return isPalindrome(s.substring(1, s.length() - 1));
}
```

RecursivePalindromeUsingSubstring

Pearson

Copyright © 2024 Pearson Education, Inc. All Rights Reserved

Recursive Helper Methods (1 of 2)

Sometimes you can find a solution by defining a recursive method to a problem similar to the original problem. This new method is called a recursive helper method. The original method can be solved by invoking the recursive helper method.

Pearson

Copyright © 2024 Pearson Education, Inc. All Rights Reserved

Recursive Helper Methods (2 of 2)

The preceding recursive isPalindrome method is not efficient, because it creates a new string for every recursive call. To avoid creating new strings, use a helper method:

```
public static boolean isPalindrome(String s) {
  return isPalindrome(s, 0, s.length() - 1);
}
public static boolean isPalindrome(String s, int low, int high) {
  if (high <= low) // Base case
    return true;
  else if (s.charAt(low) != s.charAt(high)) // Base case
    return false;
  else
    return isPalindrome(s, low + 1, high - 1);
}
```

RecursivePalindrome

Pearson

Copyright © 2024 Pearson Education, Inc. All Rights Reserved

Recursive Selection Sort

1. Find the smallest number in the list and swaps it with the first number.
2. Ignore the first number and sort the remaining smaller list recursively.

RecursiveSelectionSort

Pearson

Copyright © 2024 Pearson Education, Inc. All Rights Reserved

Recursive Binary Search

1. Case 1: If the key is less than the middle element, recursively search the key in the first half of the array.
2. Case 2: If the key is equal to the middle element, the search ends with a match.
3. Case 3: If the key is greater than the middle element, recursively search the key in the second half of the array.

RecursiveBinarySearch

Pearson

Copyright © 2024 Pearson Education, Inc. All Rights Reserved

Recursive Implementation (1 of 2)

```
/** Use binary search to find the key in the list */
public static int recursiveBinarySearch(int[] list, int key) {
  int low = 0;
  int high = list.length - 1;
  return recursiveBinarySearch(list, key, low, high);
}
```

Pearson

Copyright © 2024 Pearson Education, Inc. All Rights Reserved

Recursive Implementation (2 of 2)

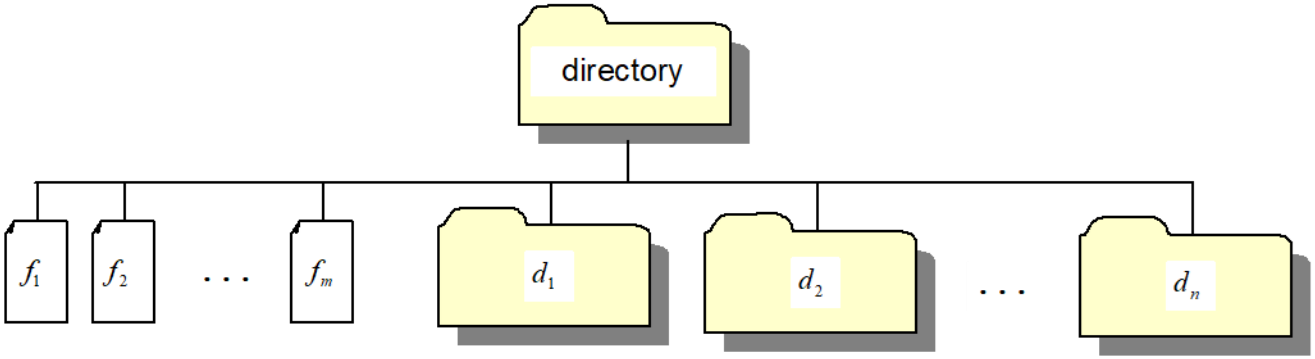
```
/** Use binary search to find the key in the list between
   list[low] list[high] */
public static int recursiveBinarySearch(int[] list, int key,
  int low, int high) {
  if (low > high) // The list has been exhausted without a match
    return -low - 1;
  int mid = (low + high) / 2;
  if (key < list[mid])
    return recursiveBinarySearch(list, key, low, mid - 1);
  else if (key == list[mid])
    return mid;
  else
    return recursiveBinarySearch(list, key, mid + 1, high);
}
```

Pearson

Copyright © 2024 Pearson Education, Inc. All Rights Reserved

Directory Size (1 of 2)

The preceding examples can easily be solved without using recursion. This section presents a problem that is difficult to solve without using recursion. The problem is to find the size of a directory. The size of a directory is the sum of the sizes of all files in the directory. A directory may contain subdirectories. Suppose a directory contains files , , …, , and subdirectories , , …, , as shown below.



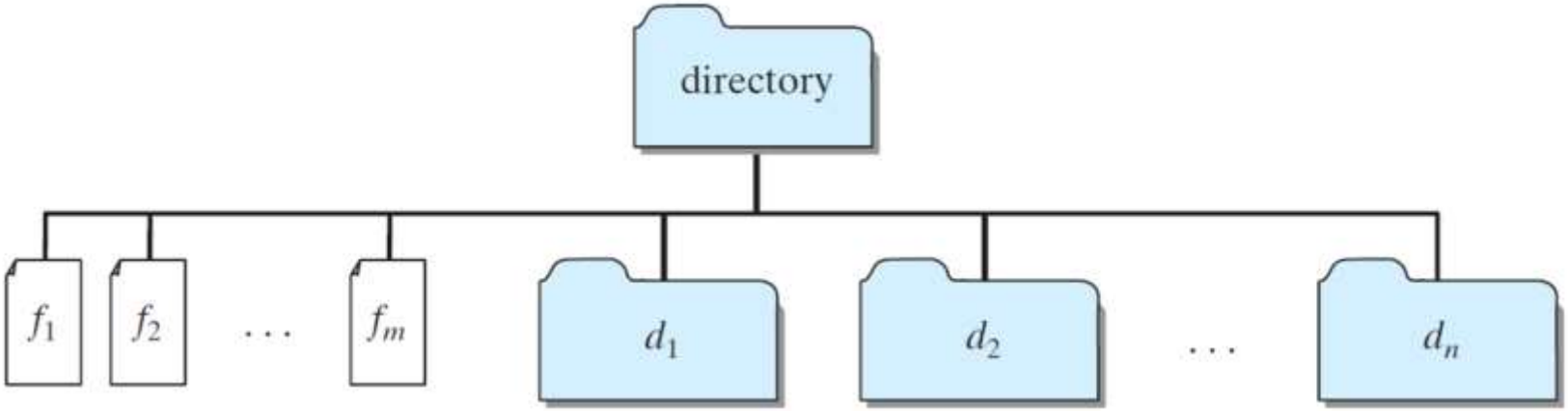
Pearson

Copyright © 2024 Pearson Education, Inc. All Rights Reserved

Directory Size (2 of 2)

The size of the directory can be defined recursively as follows:

$$size(d) = size(f_1) + size(f_2) + \ldots + size(f_m) + size(d_1) + size(d_2) + \ldots + size(d_n)$$



DirectorySize

Pearson

Copyright © 2024 Pearson Education, Inc. All Rights Reserved

Tower of Hanoi (1 of 2)

- There are *n* disks labeled 1, 2, 3, …, *n*, and three towers labeled A, B, and C.
- No disk can be on top of a smaller disk at any time.
- All the disks are initially placed on tower A.
- Only one disk can be moved at a time, and it must be the top disk on the tower.

Pearson

Copyright © 2024 Pearson Education, Inc. All Rights Reserved

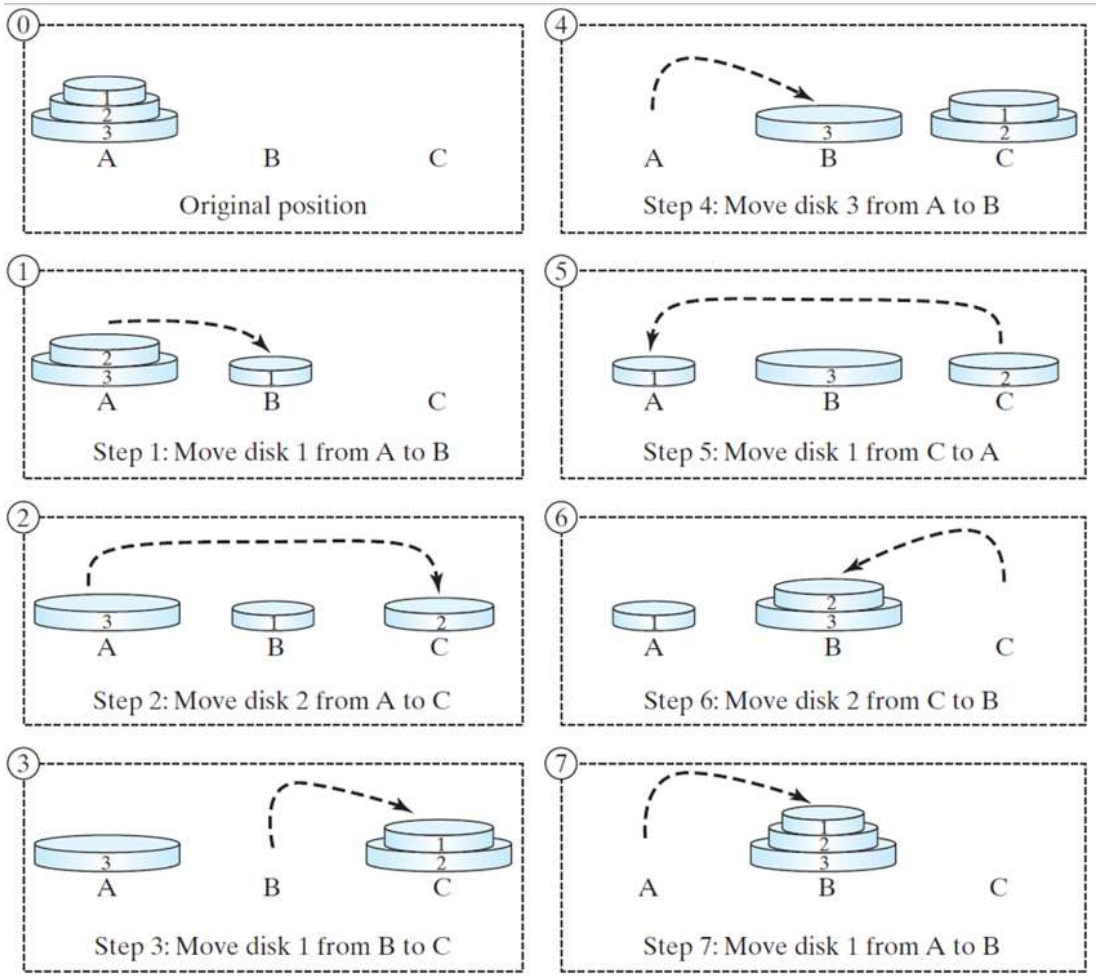
Tower of Hanoi Animation

https://liveexample.pearsoncmg.com/dsanimation/TowerOfHanoi.html

Pearson

Copyright © 2024 Pearson Education, Inc. All Rights Reserved

Tower of Hanoi (2 of 2)

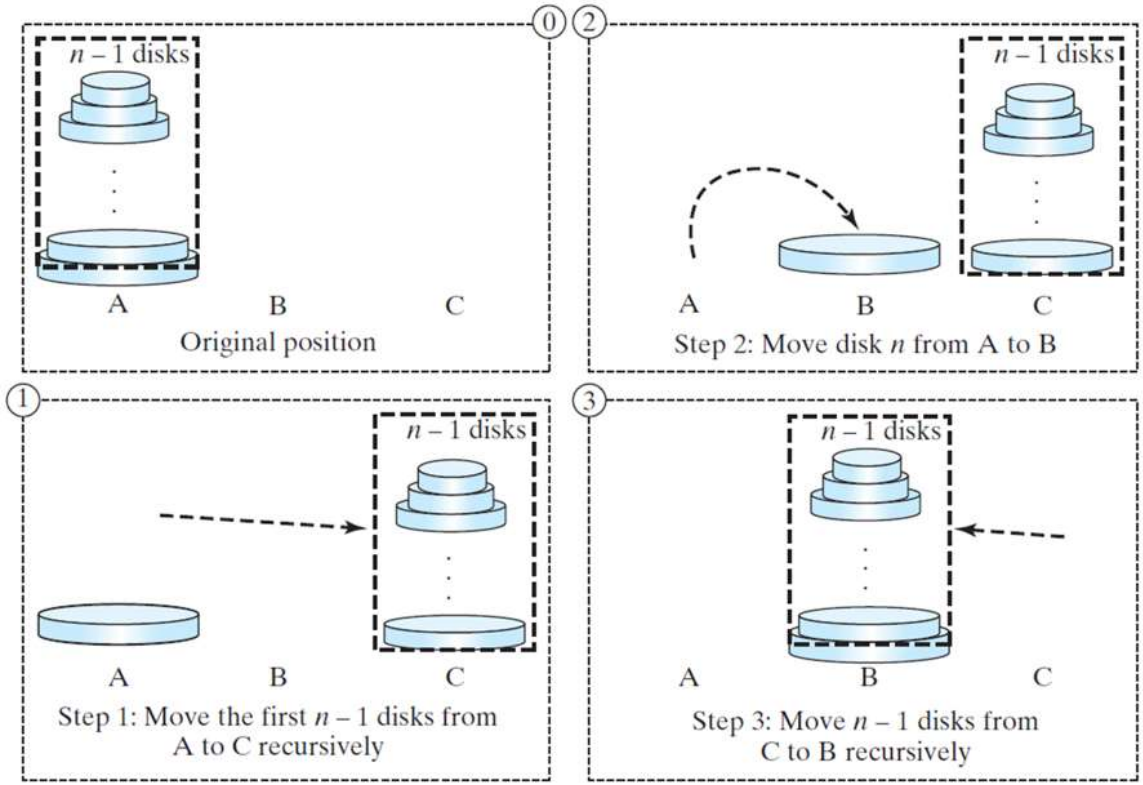


Pearson

Copyright © 2024 Pearson Education, Inc. All Rights Reserved

Solution to Tower of Hanoi (1 of 2)

The Tower of Hanoi problem can be decomposed into three subproblems.



Pearson

Copyright © 2024 Pearson Education, Inc. All Rights Reserved

Solution to Tower of Hanoi (2 of 2)

- Move the first $n-1$ disks from A to C with the assistance of tower B.
- Move disk n from A to B.
- Move $n-1$ disks from C to B with the assistance of tower A.

TowerOfHanoi

Pearson

Copyright © 2024 Pearson Education, Inc. All Rights Reserved

Exercise 18.3 GCD

```
gcd(2, 3) = 1
gcd(2, 10) = 2
gcd(25, 35) = 5
gcd(205, 301) = 5
gcd(m, n)
```

Approach 1: Brute-force, start from `min(n, m)` down to 1, to check if a number is common divisor for both m and n, if so, it is the greatest common divisor.

Approach 2: `Euclid's` algorithm

Approach 3: Recursive method

Pearson

Copyright © 2024 Pearson Education, Inc. All Rights Reserved

Approach 2: `Euclid's` Algorithm

```
// Get absolute value of m and n;
t1 = Math.abs(m); t2 = Math.abs(n);
// r is the remainder of t1 divided by t2;
r = t1 % t2;
while (r != 0) {
 t1 = t2;
 t2 = r;
 r = t1 % t2;
}

// When r is 0, t2 is the greatest common
// divisor between t1 and t2
return t2;
```

Pearson

Copyright © 2024 Pearson Education, Inc. All Rights Reserved

Approach 3: `Recursive` Method

```
gcd(m, n) = n if m % n = 0;

gcd(m, n) = gcd(n, m % n); otherwise;
```

Pearson

Copyright © 2024 Pearson Education, Inc. All Rights Reserved

Fractals

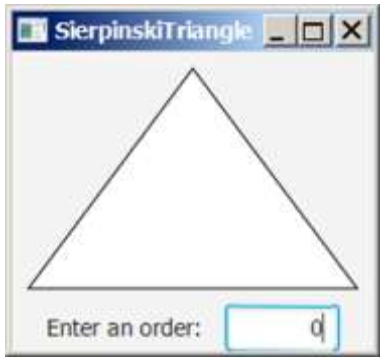
A fractal is a geometrical figure just like triangles, circles, and rectangles, but fractals can be divided into parts, each of which is a reduced-size copy of the whole. There are many interesting examples of fractals. This section introduces a simple fractal, called **Sierpinski triangle**, named after a famous Polish mathematician.

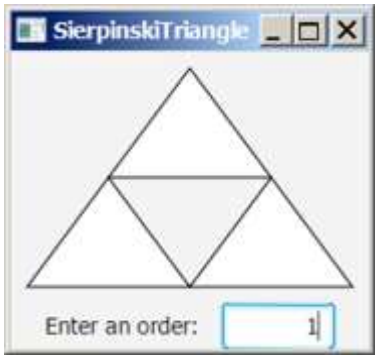
Pearson

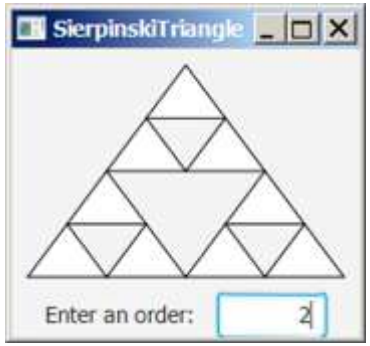
Copyright © 2024 Pearson Education, Inc. All Rights Reserved

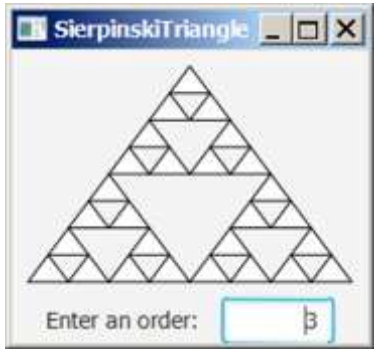
Sierpinski Triangle

1. It begins with an equilateral triangle, which is considered to be the Sierpinski fractal of order (or level) 0, as shown in Figure (a).
2. Connect the midpoints of the sides of the triangle of order 0 to create a Sierpinski triangle of order 1, as shown in Figure (b).
3. Leave the center triangle intact. Connect the midpoints of the sides of the three other triangles to create a Sierpinski of order 2, as shown in Figure (c).
4. You can repeat the same process recursively to create a Sierpinski triangle of order 3, 4, …, and so on, as shown in Figure (d).





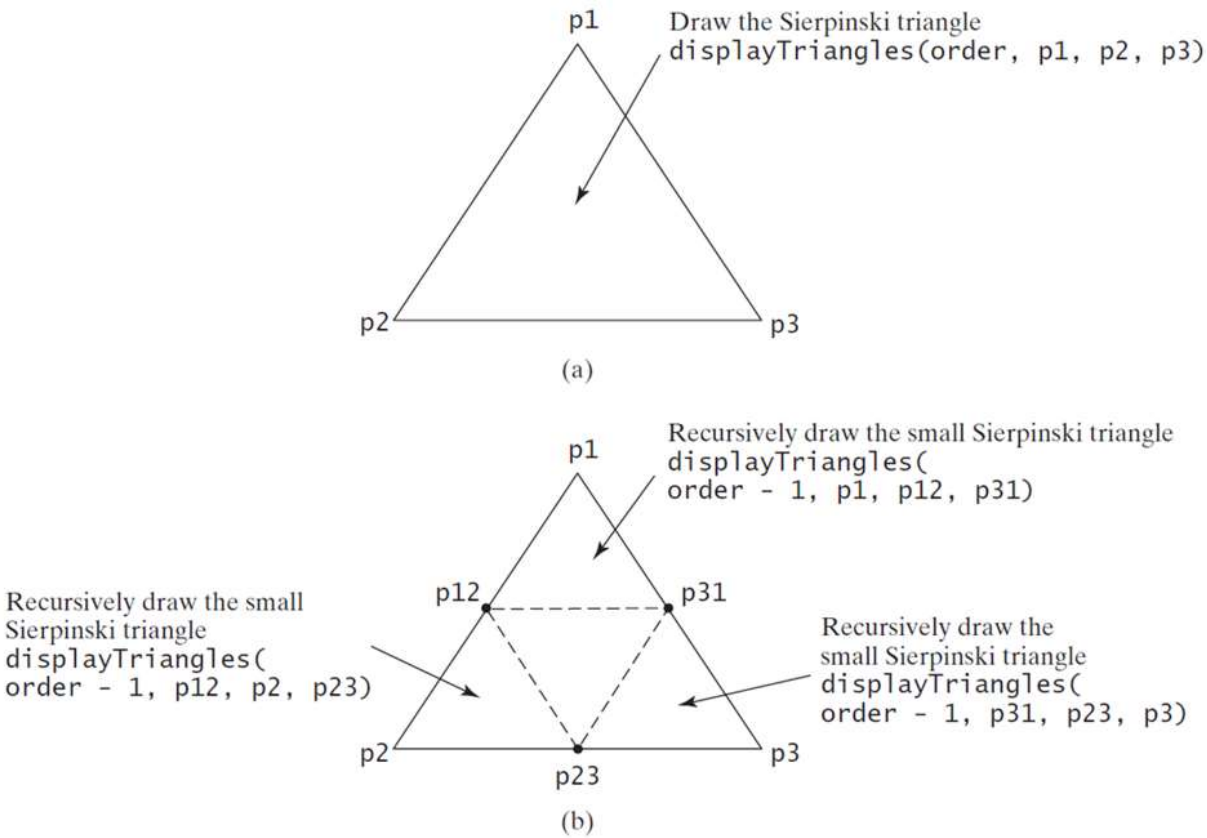




Pearson

Copyright © 2024 Pearson Education, Inc. All Rights Reserved

Sierpinski Triangle Solution



SierpinskiTriangle

Pearson

Copyright © 2024 Pearson Education, Inc. All Rights Reserved

Recursion vs Iteration

Recursion is an alternative form of program control. It is essentially repetition without a loop.

Recursion bears substantial overhead. Each time the program calls a method, the system must assign space for all of the method's local variables and parameters. This can consume considerable memory and requires extra time to manage the additional space.

Pearson

Copyright © 2024 Pearson Education, Inc. All Rights Reserved

Advantages of Using Recursion

Recursion is good for solving the problems that are inherently recursive.

Pearson

Copyright © 2024 Pearson Education, Inc. All Rights Reserved

Tail Recursion

A recursive method is said to be **tail recursive** if there are no pending operations to be performed on return from a recursive call.

Non-tail recursive	ComputeFactorial
Tail recursive	ComputeFactorialTailRecursion

Pearson

Copyright © 2024 Pearson Education, Inc. All Rights Reserved

Copyright

This work is protected by United States copyright laws and is provided solely for the use of instructors in teaching their courses and assessing student learning. Dissemination or sale of any part of this work (including on the World Wide Web) will destroy the integrity of the work and is not permitted. The work and materials from it should never be made available to students except by instructors using the accompanying text in their classes. All recipients of this work are expected to abide by these restrictions and to honor the intended pedagogical purposes and the needs of other instructors who rely on these materials.

Pearson

Copyright © 2024 Pearson Education, Inc. All Rights Reserved